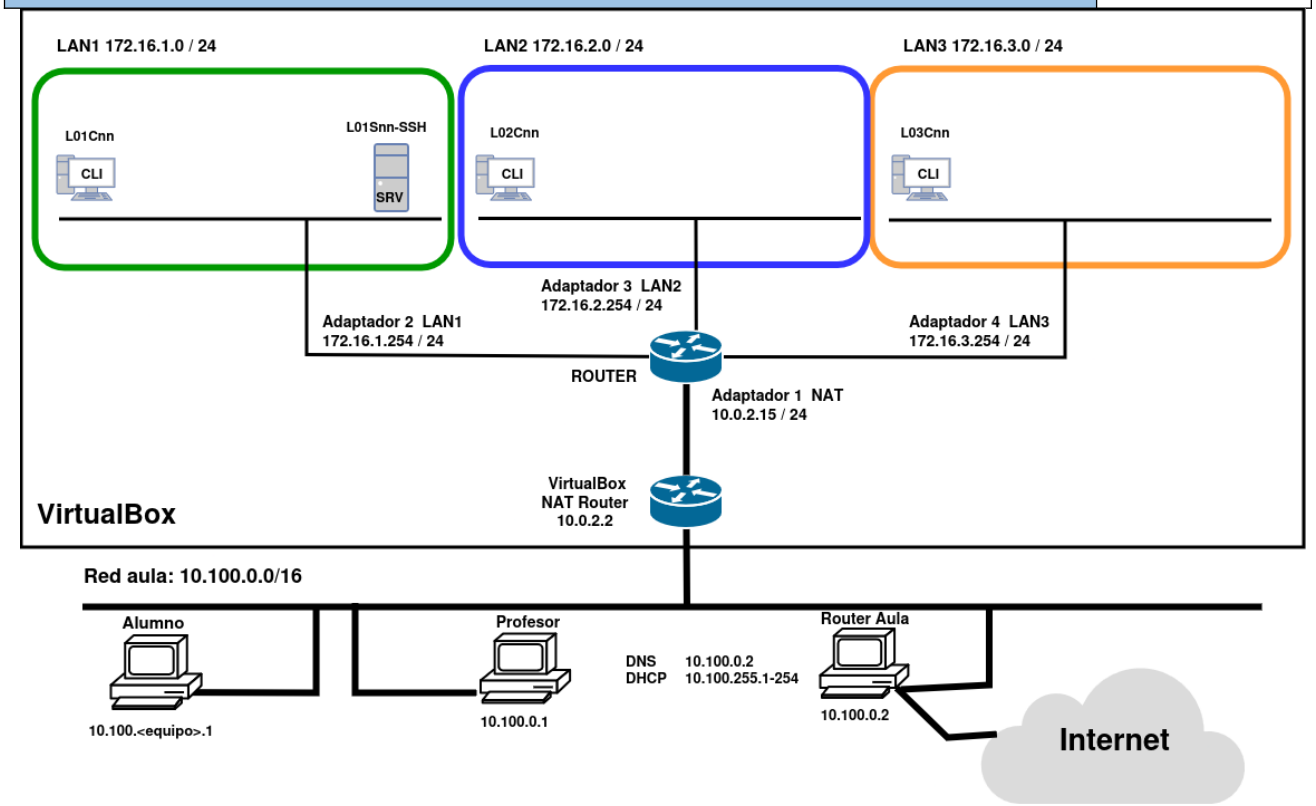
SSH LINUX OPENSSH

ACTIVIDAD



OBJETIVO

El objetivo de está práctica es aprender a realizar la configuración básica de un servidor SSH en Linux centrándonos en aspectos como la configuración del acceso autenticado y anónimo, la gestión de las diferentes operaciones permitidas a los usuarios, registro y auditoría de sucesos, las restricciones de acceso y la seguridad. Mediante un juego de pruebas extenso se reforzarán los conceptos relativos a la configuración del servicio y los conceptos teóricos del protocolo SSH.

Especificaciones

L01Snn-SSH-Linux Sistema Operativo Linux Debian Red interna LAN1 RAM 512MB	Configuración red estática IP 172.16.1.5 / 24 Gateway 172.16.1.254 DNS 10.100.0.2
L02Cnn-Alpine Sistema Operativo Linux Alpine Red interna LAN2	Configuración de red DHCP.
L03Cnn-Alpine Sistema Operativo Linux Alpine Red interna LAN3	Configuración de red DHCP.

L01Snn-SSH-Linux configuración del servicio SSH

	Grupo	Usuario	Contraseña
Grupos y Usuarios	smrA	alu1smrA	u1A
		alu2smrA	u2A
		alu3smrA	u3A
	smrB	alu1smrB	u1B
		alu2smrB	u2B
	smrD	alu1smrD	u1D
		alu2smrD	u2D
		profesor	profesor
		root	root

Restricciones			
Grupo/Usuario	Ubicación	Tiempo	Otras
smrA	172.16.2.0/24	Lunes-Martes 08:00-15:00 Miércoles 09:00-14:00 Jueves-Viernes 08:00-15:00	Acceso sólo usuario/contraseña
smrB	172.16.3.0/24	Lunes-Jueves 08:00-15:00 Viernes 08:00-13:00	
smrD	172.16.2.0/24 172.16.3.0/24	Laborables (todo el día) Sabado 08:00-14:00	Acceso usuario/contraseña Acceso certificado personal
profesor			Acceso sólo certificado personal
root			Acceso (Doble factor): certificado personal Acceso (Doble factor): Google Authenticator

1. Instalación

a) Crear la máquina virtual con los parámetros adecuados según las especificaciones

General

Nombre: L01S00-SSH-Linux
Sistema operativo: Debian 11 Bullseye (64-bit)
Grupos: SER/SERVERS

Sistema

Memoria base: 512 MB
Orden de arranque: Disquete, Óptica, Disco duro
Aceleración: VT-x/AMD-V, Paginación anidada, Paravirtualización KVM

Pantalla

Memoria de vídeo: 16 MB
Controlador gráfico: VMSVGA
Servidor de escritorio remoto: Inhabilitado
Grabación: Inhabilitado

Almacenamiento

Controlador: IDE
IDE secundario maestro: [Unidad óptica] Vacío
Controlador: SATA
Puerto SATA 0: Debian11.vdi (Normal, 20,00 GB)

Audio

Controlador de anfitrión: Predeterminado
Controlador: ICH AC97

Red

Adaptador 1: Intel PRO/1000 MT Desktop (Red interna, «LAN1»)

b) Cambiar el nombre de la máquina

Cambiar configuración prompt

Editamos el fichero de configuración del prompt y lo lo configuramos para que cuando se cambie el nombre de la máquina, automáticamente se actualice el prompt sin necesidad de hacer el *logout*.

Archivo ~/.bashrc

```
...
PS1='\u@$(hostname):\w\$ '
...
```

Cambiar nombre de la máquina (L01Snn-SSH-Linux)

```
root@debian-11:~# hostnamectl set-hostname L01S00-SSH-Linux
root@L01S00-SSH-Linux:~#
```

c) Configurar los parámetros de red

Desactivar interfaz de red

```
root@L01S00-SSH-Linux:~# ifdown enp0s3
```

Configurar ip

Editar fichero /etc/network/interfaces y configurarlo de la siguiente manera:

Archivo /etc/network/interfaces

```
...
auto enp0s3
iface enp0s3 inet static
      address 172.16.1.5
      netmask 255.255.255.0
      gateway 172.16.1.254
...
```

Configurar DNS

Archivo /etc/resolv.conf

```
nameserver 10.100.0.2
```

Activar interfaz

```
root@L01S00-SSH-Linux:~# ifup enp0s3
```

d) Instalación del servidor ssh (openssh-server).

Actualizar el sistema

```
root@L01S00-SSH-Linux:~# apt update
root@L01S00-SSH-Linux:~# apt upgrade
```

Instalar servidor SSH

```
root@L01S00-SSH-Linux:~# apt install openssh-server
```

Comprobar estado del servicio

```
root@L01S00-SSH-Linux:~# systemctl is-active ssh.service
active
```

También podríamos haber indicado sshd.service (es un alias de ssh.service).

e) Instalación del cliente ssh en cada equipo cliente (openssh-client).

Actualizar lista paquetes y software (L02Cnn-Alpine)

```
L02C00-Alpine:~# apk update
L02C00-Alpine:~# apk upgrade
```

Instalación (L02Cnn-Alpine)

```
L02C00-Alpine:~# apk add openssh-client
```

Repetir el mismo proceso en L03Cnn-Alpine

No hace falta documentarlo

f) Prueba inicial de funcionamiento

Comprobar cual es la configuración efectiva de la autenticación en el servidor

```
root@L01S00-SSH-Linux:~# sshd -T | grep -i authentication
hostbasedauthentication no
pubkeyauthentication yes
kerberosauthentication no
gssapiauthentication no
passwordauthentication yes
kbdinteractiveauthentication no
challengeresponseauthentication no
authenticationmethods any
root@L01S00-SSH-Linux:~#
```

Inicialmente nos podemos conectar utilizando tanto usuario/contraseña como con certificado personal.

Conexión desde cliente L02nn-Alpine con usuario alu1smrA

```
LO2C00-Alpine:~# ssh alu1mrA@172.16.1.5
The authenticity of host '172.16.1.5 (172.16.1.5)' can’t be established.
ED25519 key fingerprint is SHA256:9T7emIwJ61kXqqoPLnfsmN4OV5Zjw4tU2iTTqi0QIKU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.16.1.5' (ED25519) to the list of known hosts.
...
alu1mrA@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
alu1mrA@L01S00-SSH-Linux:~$
```

Vemos que tras validarnos con usuario/contraseña nos podemos conectar con el servidor SSH. Es la primera vez que nos conectamos con el servidor y éste se autentica ante nosotros instándonos a aceptar su clave pública. Otras conexiones posteriores ya no nos mostrarán ese mensaje.

2. Creación de usuarios y grupos

a) Creación de grupos y usuarios

Crear grupos

```
root@L01S00-SSH-Linux:~# groupadd smrA
root@L01S00-SSH-Linux:~# groupadd smrB
root@L01S00-SSH-Linux:~# groupadd smrD
```

Crear usuarios

```
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrA alu1smrA
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrA alu2smrA
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrA alu3smrA
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrB alu1smrB
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrB alu2smrB
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrD alu1smrD
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash -G smrD alu2smrD
root@L01S00-SSH-Linux:~# useradd -m -s /bin/bash profesor
```

Assignar contraseñas a los usuarios

```
root@L01S00-SSH-Linux:~# passwd alu1smrA
root@L01S00-SSH-Linux:~# passwd alu2smrA
root@L01S00-SSH-Linux:~# passwd alu3smrA
root@L01S00-SSH-Linux:~# passwd alu1smrB
root@L01S00-SSH-Linux:~# passwd alu2smrB
root@L01S00-SSH-Linux:~# passwd alu1smrD
root@L01S00-SSH-Linux:~# passwd alu2smrD
root@L01S00-SSH-Linux:~# passwd profesor
```

Tras la ejecución de cada comando se nos pide la contraseña y está se guardará encriptada.

b) Alternativa: script de creación de grupos y usuarios automatizada

Comprobar tipo encriptación password

Mostramos el fichero /etc/shadow y nos fijamos para un usuario (una línea) en los 3 primeros caracteres justo después del nombre del usuario.

```
root@L01S00-SSH-Linux:~# cat /etc/shadow | grep jose
jose:$y$j9T$sTnqH16t0ls2h8R0pxC8XCH.$NBG/u4526S2XiAXrBfCcXr5wmRNI0xp9I0uWNnofpRA:20501:0:99999:7:::
root@L01S00-SSH-Linux:~#
```

En este caso *y* significa encriptación *Yescrypt*, que es propia de distribuciones modernas como Debian 11 o Ubuntu 22.

Instalar herramienta encriptación

La herramienta de encriptación se llama *mkpasswd* y se encuentra en el paquete *whois* que no viene instalado por defecto

```
root@L01S00-SSH-Linux:~# apt install whois
```

Crear script

El siguiente script crea los grupos y a continuación añade los usuarios.

Extracto del archivo /usr/local/sbin/crear_usuarios.sh

```
#!/bin/bash

# Creación de grupos
groupadd smrA
groupadd smrB
groupadd smrD

# Creación de usuarios
useradd -m -G smrA -s /bin/bash -p $(mkpasswd --method=yescrypt u1A) alu1smrA
useradd -m -G smrA -s /bin/bash -p $(mkpasswd --method=yescrypt u2A) alu2smrA
useradd -m -G smrA -s /bin/bash -p $(mkpasswd --method=yescrypt u3A) alu3smrA
useradd -m -G smrB -s /bin/bash -p $(mkpasswd --method=yescrypt u1B) alu1smrB
useradd -m -G smrB -s /bin/bash -p $(mkpasswd --method=yescrypt u2B) alu2smrB
useradd -m -G smrD -s /bin/bash -p $(mkpasswd --method=yescrypt u1D) alu1smrD
useradd -m -G smrD -s /bin/bash -p $(mkpasswd --method=yescrypt u2D) alu2smrD
useradd -m -s /bin/bash -p $(mkpasswd --method=yescrypt profesor) profesor
```

Después de crear el script le tenemos que dar permisos de ejecución:

```
root@L01S00-SSH-Linux:~# chmod +x /usr/local/sbin/crear_usuarios.sh
```

Ejecutar script

```
root@L01S00-SSH-Linux:~# crear_usuarios.sh
```

Lo podemos invocar sin poner la ruta absoluta por que /usr/local/sbin se encuentra en la variable de entorno PATH.

Este script es muy sencillo y, hoy en día con la ayuda de la IA, se pueden crear scripts mucho más sofisticados para la automatización de tareas.

Se puede confirmar la creación correcta de los grupos y los usuarios con el comando **getent.**

```
root@L01S00-SSH-Linux:~# getent group smrA
smrA:x:1001:alu1smrA,alu3smrA,alu3smrA
root@L01S00-SSH-Linux:~# getent group smrB
smrB:x:1002:alu1smrB,alu2smrB
root@L01S00-SSH-Linux:~# getent group smrD
smrD:x:1003:alu1smrD,alu2smrD
root@L01S00-SSH-Linux:~# getent group smrD
smrD:x:1003:alu1smrD,alu2smrD
root@L01S00-SSH-Linux:~# getent passwd profesor
profesor:x:1008:1011::/home/profesor:/bin/bash
root@L01S00-SSH-Linux:~# getent passwd alu1smrA
alu1smrA:x:1001:1004::/home/alu1smrA:/bin/bash
...
```

3. Configurar Acceso usuarios y grupos

a) Restringir el acceso por ssh a los grupos y usuarios especificados

Restringir acceso a grupos y usuarios

```
Extracto del archivo /etc/ssh/sshd_config
...
# Permitir solo grupos smrA, smrB y smrD, y usuarios profesor y root
AllowGroups smrA smrB smrD profesor root
...
```

Permitir solo acceso autenticado

```
Extracto del archivo /etc/ssh/sshd_config
...
# PasswordAuthentication yes
...
# PubKeyAuthentication yes
PubKeyAuthentication no
...
```

Comprobar errores sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Comprobar que acceso por clave publica está deshabilitado

```
root@L01S00-SSH-Linux:~# sshd -T | grep -i PubKeyAuthentication
pubkeyauthentication no
root@L01S00-SSH-Linux:~#
```

Recargar la configuración

```
root@L01S00-SSH-Linux:~# systemctl reload ssh.service
root@L01S00-SSH-Linux:~#
```

b) Probar que solo pueden acceder los usuarios especificados. Para cada caso justificar el resultado de la operación

Acceder con el usuario alu2smrA desde L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh alu2mrA@172.16.1.5
...
alu2smrA@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
alu2mrA@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~#
```

Probar el acceso con los otros usuarios del grupo smrA y con los usuarios de los grupos smrB y smrD, pero no lo documentéis.

El usuario *alu2smrA* puede acceder por que pertenece al grupo *smrA*.

Acceder con el usuario *profesor* desde L03Cnn-Alpine

```
LO3C00-Alpine:~# ssh profesor@172.16.1.5
The authenticity of host '172.16.1.5 (172.16.1.5)' can’t be established.
ED25519 key fingerprint is: SHA256:9T7emWxuJ61kXqqoPLnfsmN4OV5Zjw4tU2iTTqi0QIKU
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.16.1.5' (ED25519) to the list of known hosts.
...
profesor@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
profesor@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO3C00-Alpine:~#
```

El usuario *profesor* puede acceder puesto que explícitamente se le permite el acceso. Realmente lo que hemos hecho es permitir el acceso al grupo unipersonal *profesor*. Si hubiéramos puesto una cláusula *AllowUsers* profesor entonces se le hubiera permitido el acceso al usuario profesor, pero se les hubiera denegado al resto de usuarios.

Acceder con el usuario *jose* desde L02Cnn-Alpine

Si no existe ese usuario en el servidor SSH, creadlo previamente

```
LO2C00-Alpine:~# ssh jose@172.16.1.5
...
jose@172.16.1.5's password:
Permission denied, please try again.
jose@172.16.1.5's password:
Permission denied, please try again.
jose@172.16.1.5's password:
jose@172.16.1.5: Permission denied (password).
LO2C00-Alpine:~#
```

Tras intentarlo hasta en 3 ocasiones (valor por defecto) el usuario *jose* no puede establecer una conexión con el servidor SSH. Es correcto pues no pertenece a ninguno de los grupos permitidos ni tampoco se le permite el acceso explícitamente.

Podemos examinar el fichero de log de auditoría para confirmar cuál ha sido la causa.

```
root@L01S00-SSH-Linux:~# cat /var/log/auth.log | grep allowed
Feb 27 10:50:53 L01S00-SSH-Linux sshd[7542]: User jose from 172.16.2.195 not allowed because none of user's groups are listed in AllowGroups
root@L01S00-SSH-Linux:~#
```

Acceder con el usuario *root* desde L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh root@172.16.1.5
...
root@172.16.1.5's password:
Permission denied, please try again.
root@172.16.1.5's password:
Permission denied, please try again.
root@172.16.1.5's password:
root@172.16.1.5: Permission denied (password).
LO2C00-Alpine:~#
```

A pesar de estar permitido explícitamente el acceso a *root* (grupo unipersonal *root*), no se puede acceder.

Si miramos el fichero log de auditoría no se ve una causa clara de por qué no puede acceder.

El problema es que por defecto no se le permite acceder al usuario root. Para permitir su acceso hay que configurar la directiva *PermitRootLoging* convenientemente;

```
PermitRootLoging yes
```

De momento no permitiremos el acceso al usuario root.

En este punto realizaremos una instantánea de nombre ***Configuración Inicial***.

4. Configurar parámetros y restricciones ubicación

a) Denegar por defecto cualquier tipo de autenticación

Configurar las opciones

```
Extracto del archivo /etc/ssh/sshd_config
...
PasswordAuthentication no
PubKeyAuthentication no
...
```

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración de autenticación

```
root@L01S00-SSH-Linux:~# sshd -T | grep -i authentication
hostbasedauthentication no
pubkeyauthentication no
kerberosauthentication no
gssapiauthentication no
passwordauthentication no
kbdinteractiveauthentication no
challengeresponseauthentication no
authenticationmethods any
root@L01S00-SSH-Linux:~#
```

b) Configurar restricciones de los usuarios del grupo smrA

Antes de empezar, colocar la directiva Include **/etc/ssh/sshd_config.d/*.conf** al final del fichero.

Crear archivo /etc/ssh/sshd_config.d/01-smrA.conf con las restricciones indicadas

```
Archivo /etc/ssh/sshd_config.d/01-smrA.conf
Match Group smrA Address 172.16.2.0/24
      PasswordAuthentication yes
      AuthenticationMethods password
```

Solo se permite utilizar como método de autenticación el usuario/contraseña (AuthenticationMethods).

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración de autenticación simulando acceso desde 172.16.2.0/24

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu2smrA,addr=172.16.2.195 |\
> grep -i Authentication|PubKeyAuthentication|AuthenticationMethods"
pubkeyauthentication no
passwordauthentication yes
authenticationmethods password
root@L01S00-SSH-Linux:~#
```

Validar la configuración simulando acceso desde cualquier otro sitio

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu3smrA | grep -i Authentication
...
pubkeyauthentication no
passwordauthentication no
...
root@L01S00-SSH-Linux:~#
```

c) Configurar restricciones de los usuarios del grupo smrB

Crear archivo /etc/ssh/sshd_config.d/02-smrB.conf con las restricciones indicadas

```
Archivo /etc/ssh/sshd_config.d/02-smrB.conf
Match Group smrA Address 172.16.3.0/24
      PasswordAuthentication yes
      AuthenticationMethods password
```

Solo se permite utilizar como método de autenticación el usuario/contraseña (AuthenticationMethods).

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración de autenticación simulando acceso desde 172.16.3.0/24

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu1smrB,addr=172.16.3.198 |\
> grep -i -E "PasswordAuthentication|PubKeyAuthentication|AuthenticationMethods"
pubkeyauthentication no
passwordauthentication yes
authenticationmethods password
root@L01S00-SSH-Linux:~#
```

Validar la configuración simulando acceso desde cualquier otro sitio

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu2smrB | grep -i Authentication
...
pubkeyauthentication no
passwordauthentication no
...
root@L01S00-SSH-Linux:~#
```

d) Configurar restricciones de los usuarios del grupo smrD

Crear archivo /etc/ssh/sshd_config.d/03-smrD.conf con las restricciones indicadas

```
Archivo /etc/ssh/sshd_config.d/03-smrD.conf
Match Group smrD Address 172.16.2.0/24, 172.16.3.0/24
      PubKeyAuthentication yes
      PasswordAuthentication yes
      AuthenticationMethods publickey password
```

Solo se permiten utilizar como métodos de autenticación la clave pública y el usuario/contraseña (AuthenticationMethods). Primero se intenta con el método del certificado personal (publickey) y si este falla se intenta con el usuario y la contraseña (password).

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración de autenticación simulando acceso desde 172.16.2.0/24

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu1smrD,addr=172.16.2.195 |\
> grep -i -E "PasswordAuthentication|PubKeyAuthentication|AuthenticationMethods"
pubkeyauthentication yes
passwordauthentication yes
authenticationmethods publickey password
root@L01S00-SSH-Linux:~#
```

Validar la configuración de autenticación simulando acceso desde 172.16.3.0/24

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu1smrD,addr=172.16.3.198 |\
> grep -i -E "PasswordAuthentication|PubKeyAuthentication|AuthenticationMethods"
pubkeyauthentication yes
passwordauthentication yes
authenticationmethods publickey password
root@L01S00-SSH-Linux:~#
```

Validar la configuración simulando acceso desde cualquier otro sitio

```
root@L01S00-SSH-Linux:~# sshd -T -C user=alu2smrD | grep -i Authentication
...
pubkeyauthentication no
passwordauthentication no
...
root@L01S00-SSH-Linux:~#
```

e) Configurar restricciones usuario profesor

Crear archivo /etc/ssh/sshd_config.d/04-smrD.conf con las restricciones indicadas

```
Archivo /etc/ssh/sshd_config.d/04-profesor.conf
Match User profesor
      PubKeyAuthentication yes
      AuthenticationMethods publickey
```

```
Solo se permite utilizar como método de autenticación el del certificado personal,
también llamado de clave pública (AuthenticationMethods).
```

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración desde cualquier otro sitio

```
root@L01S00-SSH-Linux:~# sshd -T -C user=profesor |\
> grep -i -E "PasswordAuthentication|PubKeyAuthentication"
pubkeyauthentication yes
passwordauthentication no
root@L01S00-SSH-Linux:~#
```

f) Pruebas de conectividad desde los clientes. Para cada prueba justifica el resultado

Recargar la configuración

```
root@L01S00-SSH-Linux:~# systemctl reload ssh.service
```

Acceso con el usuario *alu1smrA* desde el cliente L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh alu1smrA@172.16.1.5
...
alu1smrA@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:49:46 2026 from 172.16.2.195
alu1smrA@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~#
```

```
Acceso permitido; se cumple la regla de la cláusula Match.
```

Acceso con el usuario *alu1smrA* desde el cliente L03Cnn-Alpine

```
LO3C00-Alpine:~# ssh alu1smrA@172.16.1.5
...
alu1smrA@172.16.1.5's password:
Permission denied, please try again.
alu1smrA@172.16.1.5's password:
Permission denied, please try again.
alu1smrA@172.16.1.5's password:
alu1smrA@172.16.1.5: Permission denied ().
LO3C00-Alpine:~#
```

```
Acceso denegado; no se cumple la regla de la cláusula Match.
```

Acceso con el usuario *alu2smrB* desde el cliente L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh alu2smrB@172.16.1.5
...
alu2smrB@172.16.1.5's password:
Permission denied, please try again.
alu2smrB@172.16.1.5's password:
Permission denied, please try again.
alu2smrB@172.16.1.5's password:
alu2smrB@172.16.1.5: Permission denied ().
LO2C00-Alpine:~#
```

```
Acceso denegado; no se cumple la regla de la cláusula Match.
```

Acceso con el usuario *alu2smrB* desde el cliente L03Cnn-Alpine

```
LO3C00-Alpine:~# ssh alu2smrB@172.16.1.5
...
alu2smrB@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
alu2smrB@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO3C00-Alpine:~#
```

```
Acceso permitido; se cumple la regla de la cláusula Match.
```

Acceso con el usuario *alu2smrD* desde el cliente L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh alu2smrD@172.16.1.5
...
alu2smrD@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
alu2smrD@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~#
```

```
Acceso permitido; se cumple la regla de la cláusula Match.
```

Acceso con el usuario *alu2smrD* desde el cliente L03Cnn-Alpine

```
LO3C00-Alpine:~# ssh alu2smrD@172.16.1.5
...
alu2smrD@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 18:34:14 2026 from 172.16.2.195
alu2smrD@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO3C00-Alpine:~#
```

```
Acceso permitido; se cumple la regla de la cláusula Match.
```

Acceso con el usuario *profesor* desde el cliente L02Cnn-Alpine

```
LO2C00-Alpine:~# ssh profesor@172.16.1.5
...
profesor@172.16.1.5: Permission denied (publickey).
LO2C00-Alpine:~#
```

```
Se cumple la condición del Match pero se deniega el acceso por que solo se permite el
acceso con certificado. No puede acceder con certificado por que aún no se ha creado el
certificado del cliente.
```

Acceso con el usuario *profesor* desde el cliente L03Cnn-Alpine

```
LO3C00-Alpine:~# ssh profesor@172.16.1.5
...
profesor@172.16.1.5: Permission denied (publickey).
LO3C00-Alpine:~#
```

```
Se deniega el acceso por la misma razón que en el punto anterior.
```

En este punto realizaremos una instantánea de nombre ***Restricciones ubicación***.

5. Configurar acceso con certificado

a) Conexión con certificado usuario profesor

Crear un usuario local *profe* en la máquina L02Cnn-Alpine

```
LO2C00-Alpine:~# adduser profe
...
LO2C00-Alpine:~#
```

Entra con el nuevo usuario y crear pareja claves para el usuario *profesor*
Anexar el sufijo *_profesor* al nombre de las claves y poner como frase de paso *secreto profesor*. La palabra de paso es *secreto profesor*.

```
LO2C00-Alpine:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/profe/.ssh/id_ed25519): /home/profe/.ssh/id_ed25519_profesor
Created directory '/home/profe/.ssh'.
Enter passphrase for "/home/profe/.ssh/id_ed25519_profesor" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/profe/.ssh/id_ed25519_profesor
Your public key has been saved in /home/profe/.ssh/id_ed25519_profesor.pub
The key fingerprint is:
SHA256:jDHL+LiNRfLzQhCJF1f30djRON818X+QF0f+LJHZIZg profe@L02C00-Alpine
The key's randomart image is:
+--[ED25519 256]--+
|  .o....  ++ =+  |
|  .+.      EoooB*|
|  . .o     =o* .B|
|  .o *      . .+=|
|   o . S        =|
|   . *.         .|
|    . =          |
|     =.0         |
|      o ...      |
+----[SHA256]-----+
LO2C00-Alpine:~$
```

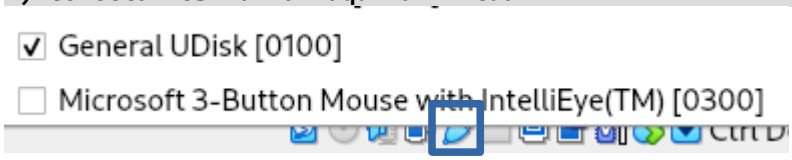
Copiar clave al servidor

```
LO2C00-Alpine:~$ ssh-copy-id -i ./.ssh/id_ed25519_profesor.pub profesor@172.16.1.5
...
profesor@172.16.1.5: Permission denied (publickey).
LO2C00-Alpine:~$
```

No se puede copiar por que no está habilitado la conexión con usuario/contraseña para el profesor.

Hay varias alternativas para solucionar el problema. Vamos a optar por una copia manual utilizando el usuario root.

Pasos:
1) Conectar USB a la máquina virtual



2) Montar el USB

```
LO2C00-Alpine:~# mount /dev/sdb1 /mnt
```

3) Copiar a la raíz (por comodidad)

```
LO2C00-Alpine:~# cp /home/profe/.ssh/id_ed25519_profesor.pub /mnt
```

4) Desmontar USB

```
LO2C00-Alpine:~$ umount /mnt
```

5) Desconectar el USB de la máquina virtual
Solo hay que desmarcar el checkbox marcado anteriormente (General Udisk [0100])

6) Conectarlo al servidor SSH
Igual que como lo hemos conectado en el cliente

7) Montar y copiar el fichero del USB y desmontar el USB

```
root@L01S00-SSH-Linux:~# mount /dev/sdb1 /mnt
root@L01S00-SSH-Linux:~# cp /mnt/id_ed25519_profesor.pub .
root@L01S00-SSH-Linux:~# umount /mnt
```

Ya podemos desconectar el USB de la máquina virtual

8) Volcar el contenido del archivo en /home/profesor/.ssh/authorized_keys

```
root@L01S00-SSH-Linux:~# mkdir /home/profesor/.ssh
root@L01S00-SSH-Linux:~# cat id_ed25519_profesor.pub > /home/profesor/.ssh/authorized_keys
root@L01S00-SSH-Linux:~# chmod 700 /home/profesor/.ssh
root@L01S00-SSH-Linux:~# chmod 600 /home/profesor/.ssh/authorized_keys
root@L01S00-SSH-Linux:~# chown -R profesor:profesor /home/profesor/.ssh
root@L01S00-SSH-Linux:~# rm id_ed25519_profesor.pub
```

Hay que crear el archivo y ajustar los permisos y el propietario tal como lo hubiera hecho el comando *ssh-copy-id*.

Ya podemos continuar desde el cliente con el usuario *profe*.

Poner en marcha el agente de claves

```
LO2C00-Alpine:~$ eval "$(ssh-agent -s)"
Agent pid 3104
LO2C00-Alpine:~$
```

Añadir clave privada al agente de claves

```
LO2C00-Alpine:~$ ssh-add ~/.ssh/id_ed25519_professor
Enter passphrase for /home/profe/.ssh/id_ed25519_profesor:
Identity added: /home/profe/.ssh/id_ed25519_profesor (profe@L02C00-Alpine)
LO2C00-Alpine:~$
```

Conectar con el usuario profesor

```
LO2C00-Alpine:~$ ssh profesor@172.16.1.5
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 22:11:46 2026
profesor@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~#
```

Conectar con usuario/contraseña

```
LO2C00-Alpine:~$ ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password profesor@172.16.1.5
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
profesor@172.16.1.5: Permission denied (publickey).
LO2C00-Alpine:~$
```

b) Conexión con certificado usuario alu1smrD

Crear un usuario local *u1smrD* en las máquinas clientes

```
LO2C00-Alpine:~# adduser u1smrD
...
LO2C00-Alpine:~#
```

Entrar con el usuario creado y crear pareja claves para el usuario alu1smrD
La palabra de paso es: *secreto alu1smrD*.

```
LO2C00-Alpine:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/u1smrD/.ssh/id_ed25519):
Enter passphrase for "/home/u1smrD/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/u1smrD/.ssh/id_ed25519
Your public key has been saved in /home/u1smrD/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:HKVSN9nJdT02DHJNSNbDNttuGxm4xPjXlHP/QQ/catrUl u1smrD@L02C00-Alpine
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|   .oB%o.        |
|    +o .*. .     |
|     .... =.o o  |
|      ..o. oo|   |
|      So  +.==|  |
|       B.0B|     |
|        * B+!    |
|         .  .!   |
+----[SHA256]-----+
LO2C00-Alpine:~$
```

Copiar clave al servidor

```
LO2C00-Alpine:~$ ssh-copy-id alu1smrD@172.16.1.5
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/u1smrD/.ssh/id_ed25519.pub"
...
alu1smrD@172.16.1.5's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'alu1smrD@172.16.1.5'"
and check to make sure that only the key(s) you wanted were added.

LO2C00-Alpine:~$
```

Poner en marcha el agente de claves

```
LO2C00-Alpine:~$ eval "$(ssh-agent -s)"
Agent pid 2460
LO2C00-Alpine:~$
```

Añadir clave privada al agente de claves

```
LO2C00-Alpine:~$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/u1smrD/.ssh/id_ed25519:
Identity added: /home/u1smrD/.ssh/id_ed25519 (u1smrD@L02C00-Alpine)
LO2C00-Alpine:~$
```

Conectar con el usuario *alu1smrD* usando el certificado

```
LO2C00-Alpine:~$ ssh alu1smrD@172.16.1.5
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 19:51:37 2026 from 172.16.2.195
alu1smrD@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~$
```

Conectar con el usuario *alu1smrD* sin usar el certificado personal

```
LO2C00-Alpine:~$ ssh -o PubkeyAuthentication=no -o PreferredAuthentications=password alu1smrD@172.16.1.5
...
alu1smrD@172.16.1.5's password:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64
...
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 19:51:37 2026 from 172.16.2.195
alu1smrD@L01S00-SSH-Linux:~$ exit
Logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~$
```

Esto demuestra que el usuario puede conectarse al servidor ssh tanto utilizando su certificado personal como su usuario/contraseña.

Se pueden hacer las mismas comprobaciones desde el equipo L00Cnn-Alpine y volver a repetir los mismos pasos para el usuario alu2smrD.

c) Conexión con certificado usuario alu3smrA (L02C00-Alpine)

Crear usuario local u3A

```
LO2C00-Alpine:~# adduser u3A
...
LO2C00-Alpine:~#
```

Entrar con el usuario creado y crear pareja claves para el usuario *alu3smrA*

```
LO2C00-Alpine:~$ ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/u3A/.ssh/id_ed25519):
Created directory '/home/u3A/.ssh'.
Enter passphrase for "/home/u3A/.ssh/id_ed25519" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/u3A/.ssh/id_ed25519
Your public key has been saved in /home/u3A/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:nbvFUdi0i0kl0wUs0kC5SgIo/aespzLGD91CUKcQuLlf84 u3A@L02C00-Alpine
The key's randomart image is:
+--[ED25519 256]--+
|                 |
|     ....  =%.   |
|   .  .  oo=.    |
| . O . . ++. .   |
| . + . . .=.     |
| . o o + S =.    |
| . . = + .       |
| ....B .         |
| . .0..E         |
|  . **           |
+----[SHA256]-----+
LO2C00-Alpine:~$
```

Copiar clave al servidor

```
LO2C00-Alpine:~$ ssh-copy-id alu3smrA@172.16.1.5
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/u3A/.ssh/id_ed25519.pub"
...
alu3smrA@172.16.1.5's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'alu3smrA@172.16.1.5'"
and check to make sure that only the key(s) you wanted were added.

LO2C00-Alpine:~$
```

Poner en marcha el agente de claves

```
LO2C00-Alpine:~$ eval "$(ssh-agent -s)"
Agent pid 2585
LO2C00-Alpine:~$
```

Añadir clave privada al agente de claves

```
LO2C00-Alpine:~$ ssh-add ~/.ssh/id_ed25519
Enter passphrase for /home/u3A/.ssh/id_ed25519:
Identity added: /home/u3A/.ssh/id_ed25519 (u3A@L02C00-Alpine)
LO2C00-Alpine:~$
```

Conectar con el usuario *alu3smrA*

```
LO2C00-Alpine:~$ ssh alu3smrA@172.16.1.5
...
alu1smrD@172.16.1.5's password:_
```

```
Ignora el certificado y nos pide la contraseña porque está deshabilitada la autenticación con certificado.
```

En este punto realizaremos una instantánea de nombre ***Configuración Acceso Certificados***.

6. Configurar restricciones temporales

a) Habilitar el uso de la librería PAM de control de acceso temporal

Comprobar si PAM está habilitado (habilitarlo si no lo estuviera)

```
root@L01S00-SSH-Linux:~# ssh -T | grep -i UsePAM
usepam yes
root@L01S00-SSH-Linux:~#
```

```
Podemos ver que PAM está habilitado por defecto.
```

Configuración para que el servicio SSH utilice la librería pam_time

Se tiene que añadir la línea mostrada:

Extracto archivo /etc/pam.d/sshd

```
...
# Control de acceso por tiempo
account required pam_time.so
...
```

b) Definir los grupos en /etc/netgroup

Crear el fichero /etc/netgroup con el siguiente contenido:

Archivo /etc/netgroup

```
smrA (,alu1smrA,) (,alu2smrA,) (,alu3smrA,)
smrB (,alu1smrB,) (,alu2smrB,)
smrD (,alu1smrD,) (,alu2smrD,)
```

Modificar la línea del archivo /etc/switch.conf de la siguiente manera:

Extracto del archivo /etc/nsswitch.conf

```
netgroup:    files nis
```

c) Definir reglas de acceso temporal

Añadir las reglas de acceso temporal según las especificaciones

Extracto archivo /etc/security/time.conf

```
...
# Grupo smrA
sshd;*;@smrA;MoTuThFr0800-1500|We0900-1400

# Grupo smrB
sshd;*;@smrB;MoTuWeTh0800-1500|Fr0800-1300

# Grupo smrD
sshd;*;@smrD;Wk0000-2400|Sa0800-1400
```

d) Preparar software de ayuda a validar restricciones horarias

Instalar faketime

```
root@L01S00-SSH-Linux:~# apt install faketime
```

Configurar creación permanente carpeta /run/sshd

Crear el archivo /etc/tmpfiles.d/sshd.conf con el siguiente contenido:

Extracto archivo /etc/tmpfiles.d/sshd.conf

```
d /run/sshd 0755 root root -
```

Creación inmediata de la carpeta /run/sshd

```
root@L01S00-SSH-Linux:~# systemd-tmpfiles --create
```

Parar el servidor ssh

```
root@L01S00-SSH-Linux:~# systemctl stop ssh.service
```

e) Validar reglas smrA

Iniciar el servidor ssh mediante faketime con un tiempo permitido para el grupo smrA

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 10:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Comprobar que puede conectar

```
LO2C00-Alpine:~$ ssh alu1smrA@172.16.1.5
...
alu1smrA@172.16.1.5's password:
...
Last login: Fri Feb 27 18:16:34 2026 from 172.16.2.195
alu1smrA@L01S00-SSH-Linux:~$
```

Mensaje en el servidor

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 10:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Accepted password for alu1smrA from 172.16.2.195 port 58428 ssh2
_
```

Desconectar el cliente

```
alu1smrA@L01S00-SSH-Linux:~$ exit
logout
Connection to 172.16.1.5 closed.
LO2C00-Alpine:~$
```

En el servidor, matar el proceso faketime (Ctrl+C) cuando el cliente desconecte

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 10:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Accepted password for alu1smrA from 172.16.2.195 port 58428 ssh2
Received disconnect from 172.16.2.195 port 58428:11: disconnected by user
Disconnected from user alu1smrA 172.16.2.195 port 58428
^C
root@L01S00-SSH-Linux:~#
```

Iniciar el servidor ssh mediante faketime con un tiempo no permitido para el grupo smrA

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-04 15:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Conectar desde el cliente

```
LO2C00-Alpine:~$ ssh alu1smrA@172.16.1.5
...
alu1smrA@172.16.1.5's password:
Connection closed by 172.16.1.5 port 22
LO2C00-Alpine:~$
```

En el servidor

```
root@L01S00-SSH-Linux:~#
root@L01S00-SSH-Linux:~# faketime -m '2026-03-04 15:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Failed password for alu1smrA from 172.16.2.195 port 49102 ssh2
Access denied for user alu1smrA by PAM account configuration [preauth]
^C
root@L01S00-SSH-Linux:~#
```

f) Validar reglas smrB

Iniciar el servidor ssh mediante faketime con un tiempo permitido para el grupo smrB

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 10:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Comprobar que puede conectar (salir después de conectar)

```
LO3C00-Alpine:~$ ssh alu2smrB@172.16.1.5
...
alu2smrB@172.16.1.5's password:
...
Last login: Fri Feb 27 18:30:06 2026 from 172.16.3.198
alu2smrB@L01S00-SSH-Linux:~$ exit
logout
Connection to 172.16.1.5 closed.
LO3C00-Alpine:~$
```

Mensaje de comprobación en el servidor. Matar el proceso cuando el cliente desconecte

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 10:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Accepted password for alu2smrB from 172.16.3.198 port 40454 ssh2
Received disconnect from 172.16.3.198 port 40454:11: disconnected by user
Disconnected from user alu2smrB 172.16.3.198 port 40454
^C
root@L01S00-SSH-Linux:~#
```

Iniciar el servidor ssh mediante faketime con un tiempo no permitido para el grupo smrB

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-06 13:30' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Conectar desde el cliente

```
LO3C00-Alpine:~$ ssh alu2smrB@172.16.1.5
...
alu2smrB@172.16.1.5's password:
Connection closed by 172.16.1.5 port 22
LO3C00-Alpine:~$
```

En el servidor

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-06 13:30' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Failed password for alu2smrB from 172.16.3.198 port 50848 ssh2
Access denied for user alu2smrB by PAM account configuration [preauth]
^C
root@L01S00-SSH-Linux:~#
```

g) Validar reglas smrD

Iniciar el servidor ssh mediante faketime con un tiempo permitido para el grupo smrD

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 21:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Comprobar que puede conectar (salir después de conectar)

```
LO3C00-Alpine:~$ ssh alu2smrD@172.16.1.5
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 20:44:20 2026 from 172.16.3.198
alu2smrD@L01S00-SSH-Linux:~$ exit
logout
Connection to 172.16.1.5 closed.
LO3C00-Alpine:~$
```

Mensaje de comprobación en el servidor. Matar el proceso cuando el cliente desconecte

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-03 21:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Accepted publickey for alu2smrD from 172.16.3.198 port 60104 ssh2: ED25519
SHA256:6NkZuC9LND5fEvAJMDPmECVa6nw0HVyACiBEUzbb7W0
Received disconnect from 172.16.3.198 port 60104:11: disconnected by user
Disconnected from user alu2smrD 172.16.3.198 port 60104
^C
root@L01S00-SSH-Linux:~#
```

Iniciar el servidor ssh mediante faketime con un tiempo no permitido para el grupo smrD

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-01 11:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
_
```

Conectar desde el cliente

```
LO3C00-Alpine:~$ ssh alu2smrD@172.16.1.5
...
Connection closed by 172.16.1.5 port 22
LO3C00-Alpine:~$
```

En el servidor

```
root@L01S00-SSH-Linux:~# faketime -m '2026-03-01 11:00' /usr/sbin/sshd -D -e
Server listening on 0.0.0.0 port 22.
Server listening on :: port 22.
Failed password for alu2smrD from 172.16.3.198 port 50848 ssh2
Access denied for user alu2smrD by PAM account configuration [preauth]
^C
root@L01S00-SSH-Linux:~#
```

En este punto realizaremos una instantánea de nombre ***Configuración Restricciones Horarias***.

7. Configurar doble factor autenticación (root)

a) Instalar y configurar aplicación Google Authenticator

Instalar paquete

```
root@L01S00-SSH-Linux:~# apt install libpam-google-authenticator
```

Ejecutar aplicación

Antes de ejecutar la aplicación es conveniente fijar un tamaño de la letra de la consola pequeño ya que si no el código QR que nos genera la aplicación nos aparecerá cortado y no lo podremos escanear.
Esta configuración del tamaño de la letra es una buena opción:

```
root@L01S00-SSH-Linux:~# setfont /usr/share/consolefonts/Lat15-Terminus12x6.psf.gz
```

Ejecutar el programa google-authenticator

Responderemos Sí (y) a todas las preguntas.

```
root@L01S00-SSH-Linux:~# google-authenticator

Do you want authentication tokens to be time-based (y/n) y
```

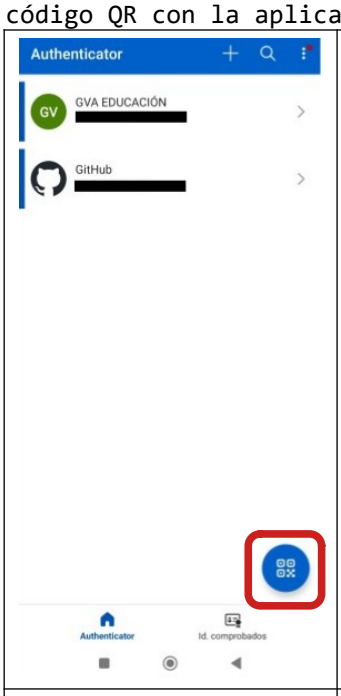
Abrir la aplicación de autenticación de vuestro móvil y escanear el código QR mostrado

En este ejemplo utilizamos Microsoft Authenticator

```
Warning: Pasting the following URL into your browser exposes the OTP secret to Google:
https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=otpauth://totp/
root@L01S00-SSH-Linux%3Fsecret%3D6PILRPRD4LUAENPSBDCTCDTCE%26issuer%3DL01S00-SSH-Linux
```

```
Your new secret key is: 6PILRPRD4LUAENPSBDCTCDTCE
Enter code from app (-1 to skip):
```

La siguiente secuencia de capturas de pantallas muestra lo que sucede tras escanear el código QR con la aplicación *Microsoft Authenticator*:

			
Selecciono la opción marcada para escanear el código QR.	Escaneo el código QR.	Automáticamente se me ha creado la cuenta a la cual se me enviarán los códigos OTP. Selecciono la nueva cuenta.	Introduzco el código mostrado en la pantalla de configuración de google-authenticator para continuar.

*** En el anexo de PAM se explica como proseguir en la configuración de google-authenticator si no podemos escanear el código QR.*

Tras introducir el código que nos da la aplicación, respondemos con Sí (y) a todas las opciones:

```
Your new secret key is: 6PILRPRD4LUAENPSBDCTCDTCE
Enter code from app (-1 to skip): 732370
Code confirmed
Your emergency scratch codes are:
  38118339
  62819937
  22053162
  66425339
  57625454

Do you want me to update your "/root/.google_authenticator" file? (y/n) y

Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y
```

```
By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
Do you want to do so? (y/n) y

If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y

root@L01S00-SSH-Linux:~#
```

b) Configurar SSH para que use la librería de autenticación de google

Configurar el archivo PAM de ssh para validarse con Google Authenticator
Se tiene que añadir las líneas mostradas:

Extracto archivo /etc/pam.d/sshd

```
...
# Autenticacion Google Authenticator

# Si el usuario NO es root, saltar las 2 lineas siguientes y continuar con common-auth
auth    [success=2 default=ignore] pam_succeed_if.so user != root

# Para root: exigir Google Authenticator (OTP)
auth    required pam_google_authenticator.so debug

# Para root: si GA tuvo exito, terminar autenticacion (no pedir password)
auth    [success=done default=ignore] pam_permit.so

# Para no-root: pila habitual (password, etc.)
@include common-auth
...
```

Si no es el usuario *root* procedemos con la validación estándard definida en el fichero */etc/ssh/pam.d/common-auth*.
Si el usuario es *root* exigimos que se valide sólo con *Google Authenticator* y acabamos el proceso de validación.

c) Configurar restricciones usuario root

Crear archivo /etc/ssh/sshd_config.d/05-root.conf con las restricciones

Archivo /etc/ssh/sshd_config.d/05-root.conf

```
Match User root
      PermitRootLogin yes
      PubkeyAuthentication yes
      AuthenticationMethods publickey,keyboard-interactive:pam
```

Comprobar si hay errores de sintaxis

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Validar la configuración desde cualquier sitio

```
root@L01S00-SSH-Linux:~# sshd -T -C user=root | grep -i authentication
hostbasedauthentication no
pubkeyauthentication yes
kerberosauthentication no
gssapiauthentication no
passwordauthentication no
kbdinteractiveauthentication yes
challengeresponseauthentication no
authenticationmethods publickey,keyboard-interactive:pam
root@L01S00-SSH-Linux:~#
```

d) Modificar restricciones resto de usuarios y grupos para que no puedan validarse con ningún método que necesita introducir datos desde el teclado, como Google Authenticator.

Modificación en archivos de los otros usuarios y grupos

Hay que añadir una cláusula KbdInteractiveAuthentication con valor no. Por ejemplo, en el archivo correspondiente al grupo smrA tendríamos lo siguiente:

```
Archivo /etc/ssh/sshd_config.d/01-smrA.conf
Match Group smrA Address 172.16.2.0/24
      PasswordAuthentication yes
      KbdInteractiveAuthentication no
      AuthenticationMethods password
```

A continuación comprobar que no hay errores de sintaxis. Posteriormente es conveniente comprobar la configuración efectiva con el comando *sshd -T* como hicimos anteriormente (no documentarlo).

```
root@L01S00-SSH-Linux:~# sshd -t
root@L01S00-SSH-Linux:~#
```

Recargar la configuración

```
root@L01S00-SSH-Linux:~# systemctl reload ssh.service
```

e) Crear certificado personal usuario root

Llevar a cabo todas estas acciones al cliente L02Cnn-Alpine

Crear pareja claves para el usuario *root*

Anexar el sufijo *_root* al nombre de las claves y poner como frase de paso *secreto root*.

```
L02C00-Alpine:~# ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (/root/.ssh/id_ed25519): /root/.ssh/id_ed25519_root
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_ed25519_root
Your public key has been saved in /root/.ssh/id_ed25519_root.pub
The key fingerprint is:
SHA256:gGPQlK5tQMQCQRKBSVULtrKCPdBrr0nh30jPUhWusnI root@L02C00-Alpine
The key's randomart image is:
+--[ED25519 256]--+
|B=oo.  .         |
|oo+..+*+         |
| + o .o..        |
|.o  . .o         |
| .*  .o$         |
|!* %.o           |
|.. . +           |
|+.o+.            |
|oo+ .            |
+----[SHA256]-----+
L02C00-Alpine:~#
```

Copiar clave pública al servidor

Como la authenticación por usuario/contraseña está deshabilitada tenemos que copiar la clave al servidor por otra vía. Vamos a probar con scp y si no se pudiera, lo tendremos que hacer manualmente.

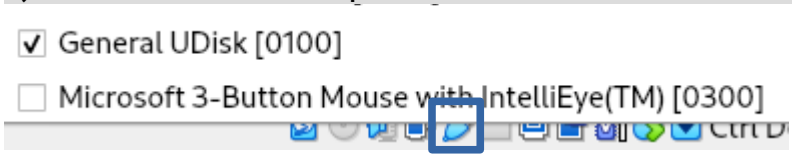
Copiar clave al servidor

```
L02C00-Alpine:~# scp ~/.ssh/id_ed25519_root root@172.16.1.5:/root
*** WARNING: connection is not using a post-quantum key exchange algorithm.
*** This session may be vulnerable to "store now, decrypt later" attacks.
*** The server may need to be upgraded. See https://openssh.com/pg.html
root@172.16.1.5: Permission denied (publickey).
scp: Connection closed
L02C00-Alpine:~#
```

No se puede copiar por que no está habilitado la conexión con usuario/contraseña para root. Por lo tanto, vamos a copiar la clave pública manualmente:

Pasos:

1) Conectar USB a la máquina virtual



2) Montar el USB

```
LO2C00-Alpine:~# mount /dev/sdb1 /mnt
```

3) Copiar a la raíz (por comodidad)

```
LO2C00-Alpine:~# cp /root/.ssh/id_ed25519_root.pub /mnt
```

4) Desmontar USB

```
LO2C00-Alpine:~# umount /mnt
```

5) Desconectar el USB de la máquina virtual

Solo hay que desmarcar el checkbox marcado anteriormente (General Udisk [0100])

6) Conectarlo al servidor SSH

Igual que como lo hemos conectado en el cliente

7) Montar y copiar el fichero del USB y desmontar el USB

```
root@L01S00-SSH-Linux:~# mount /dev/sdb1 /mnt
root@L01S00-SSH-Linux:~# cp /mnt/id_ed25519_root.pub .
root@L01S00-SSH-Linux:~# umount /mnt
```

Ya podemos desconectar el USB de la máquina virtual

8) Volcar el contenido del archivo en /root/.ssh/authorized_keys

```
root@L01S00-SSH-Linux:~# mkdir /root/.ssh
root@L01S00-SSH-Linux:~# cat id_ed25519_root.pub > /root/.ssh/authorized_keys
root@L01S00-SSH-Linux:~# rm id_ed25519_root.pub
```

Ya podemos continuar desde el cliente con el usuario *root*.

Iniciar el agente de claves

```
LO2C00-Alpine:~# eval "$(ssh-agent -s)"
Agent pid 2476
LO2C00-Alpine:~#
```

Añadir clave privada al agente de claves

```
LO2C00-Alpine:~# ssh-add ~/.ssh/id_ed25519_root
Enter passphrase for /root/.ssh/id_ed25519_root:
Identity added: /root/.ssh/id_ed25519_root (root@L02C00-Alpine)
LO2C00-Alpine:~#
```

f) Probar la conexión con doble factor de autenticación

Conectar al servidor SSH como root desde L02C00-Alpine

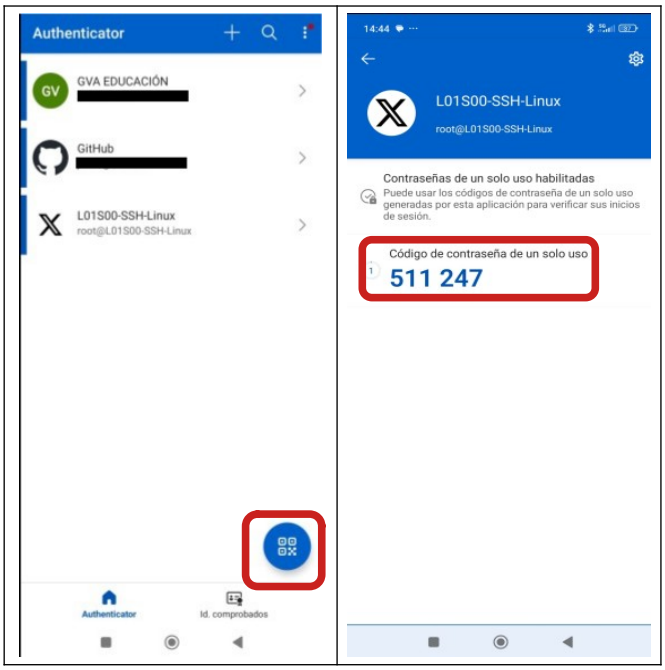
```
LO2C00-Alpine:~# ssh root@172.16.1.5
...
(root@172.16.1.5) Verification code:
Linux LO1S00-SSH-Linux 5.10.0-38-amd64 #1 SMP Debian 5.10.249-1 (2026-02-10) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Mar  1 14:49:13 2026 from 172.16.2.195
root@LO1S00-SSH-Linux:~#
LO2C00-Alpine:~#
```

Podemos ver que nos exige un código de verificación.

Entramos en nuestra aplicación de autenticación en el móvil y vemos cual es ese código (en este caso es 511247).



Tras introducir el código conectamos con el servidor SSH.

Conectar al servidor SSH como root desde L03C00-Alpine

```
LO3C00-Alpine:~# ssh root@172.16.1.5
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
root@172.16.1.5: Permission denied (publickey).
LO3C00-Alpine:~#
```

```
Se puede ver que no podemos conectar. Eso es debido a que en esta máquina no tenemos el certificado personal para el usuario root. Habría que traerse las claves que se generaron en la otra máquina a la carpeta ~/.ssh de la máquina local.
Que no se pueda conectar demuestra que, efectivamente, antes de utilizar la autenticación provista por Google Authenticator se está autenticando utilizando el certificado digital.
```

En este punto realizaremos una instantánea de nombre ***Práctica completada***.